Computational Geometry: Orthogonal Searching and Range tree

Pritha Banerjee

Resources

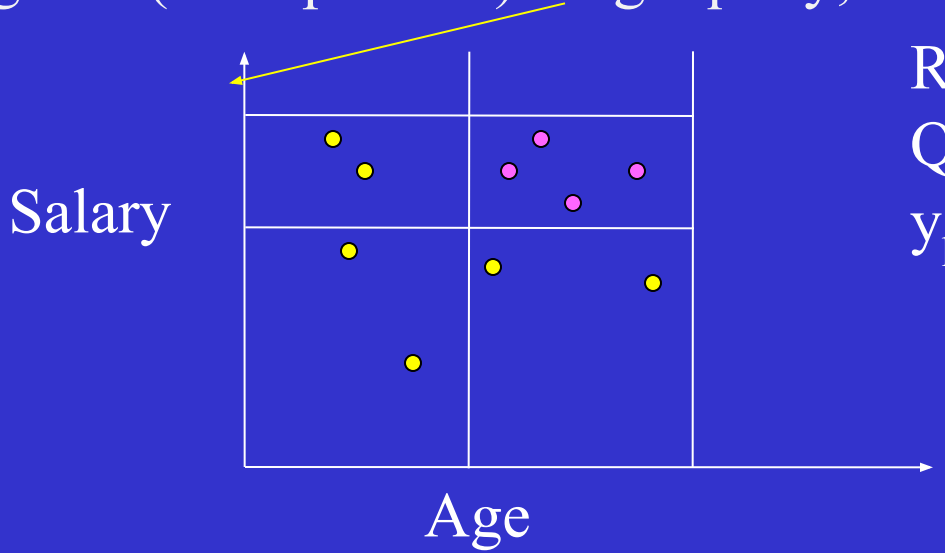
- Books and Lecture Notes:
  - Chapter 5 of Computational Geometry : Algorithms and Applications, Mark De Berg, Marc van Kreveld, Mark Overmars, Otfried Schwarzkopf, Springer
  - Lecture notes of David Mount from the web

Orthogonal Range Searching (querying a database)

Input : a query range and a set of points P

Output: Report all the points in the given range

Range query: 1 dimensional, 2 dimensional rectangular or orthogonal (axis parallel) range query, d -dimensional



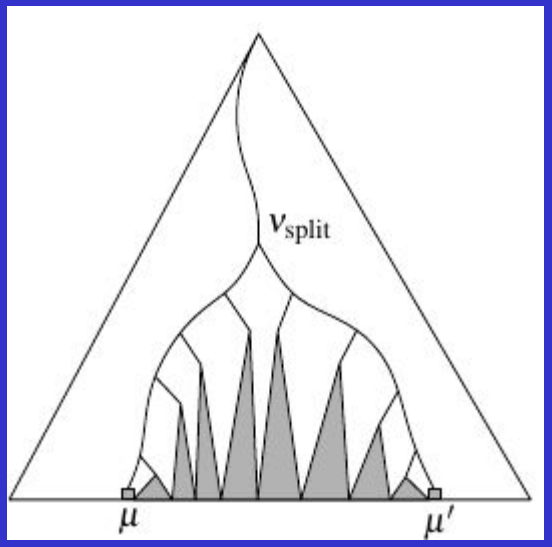
Rectangular range Query $[x_l : x_h][y_l : y_h]$

Query using Range Trees

- Time Complexity : $O(\log^2 n + k)$
- Space Complexity: $O (n \log n)$

- Idea
  - First find points in $[x_l : x_h]$: 1D range query
  - Find points whose y coordinate lies in $[y_l : y_h]$: 1D range query
- What is the data structure?
  - Range Trees

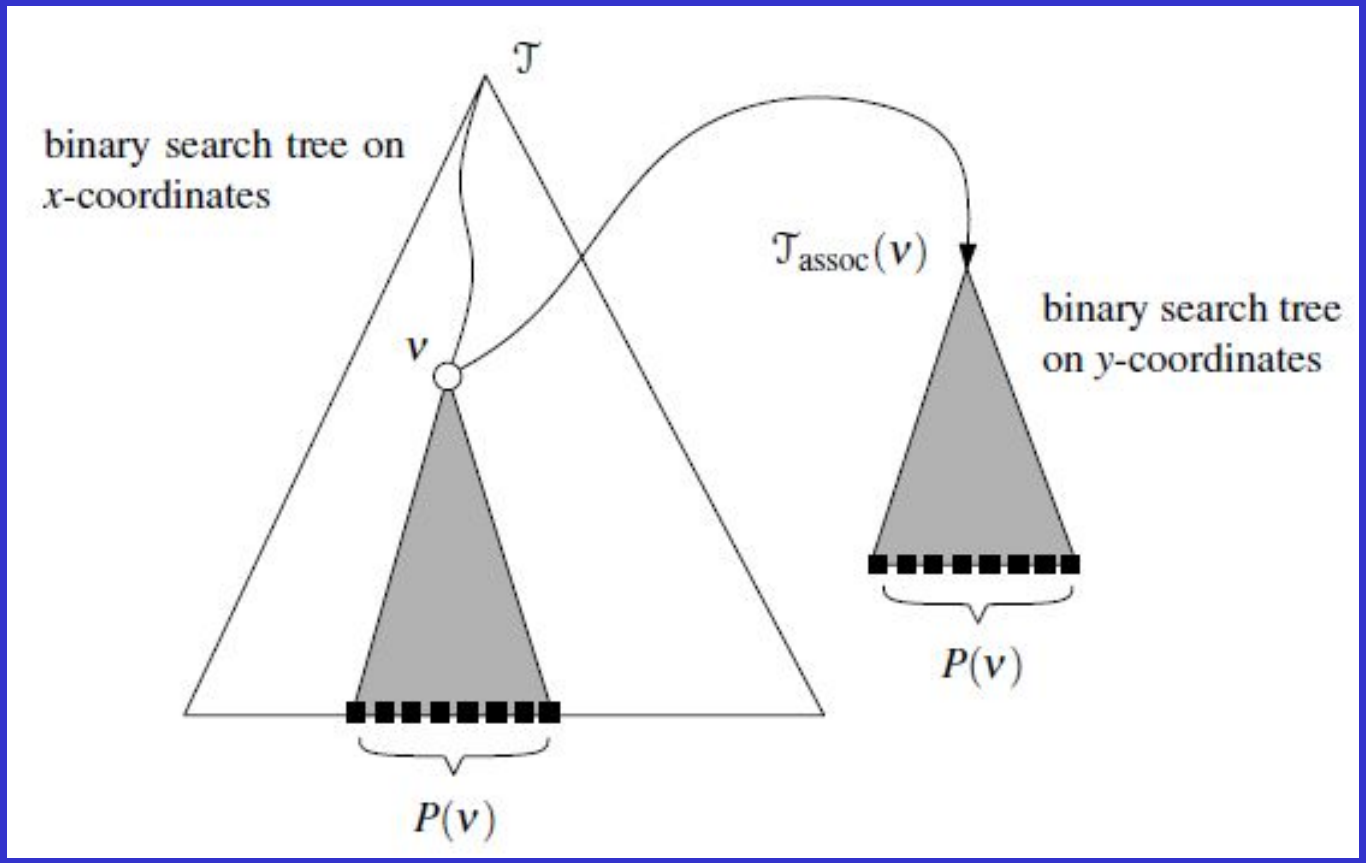
Observations: 1D range query



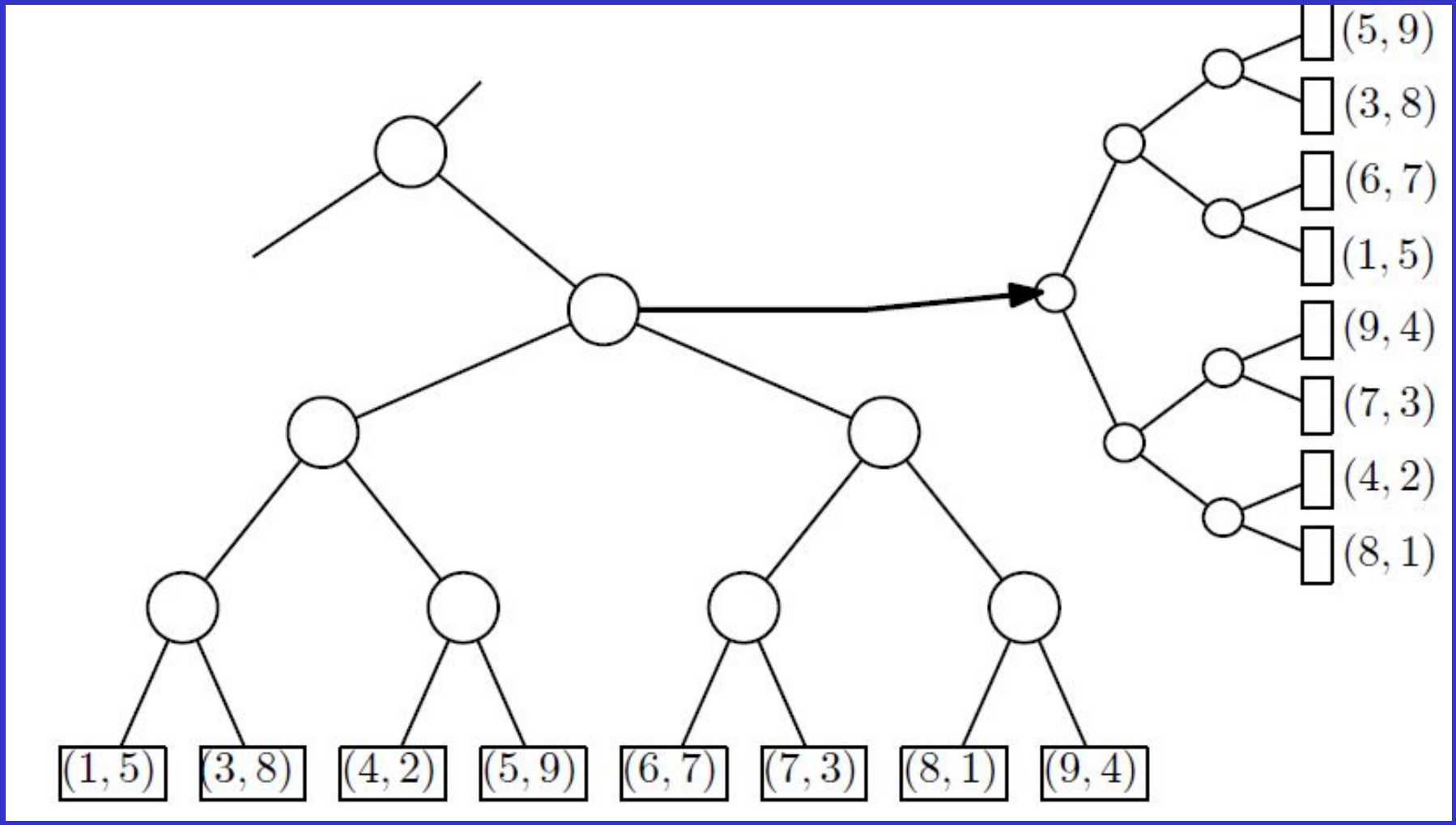
- 1D range query uses BST
- Select O(log n) subtrees that contain points in range $[x_l : x_h]$
- Canonical subset of v : subset of points stored in leaves of subtree rooted at v: P(v)
- Canonical subset of root: point set P
- Canonical subset of leaf: point stored in leaf
- Need to report subset of points in P(v) whose y co-ordinate lies in [yl : yh]
    - Need BST on y coordinates of points in P(v)

Range Tree

- Multi-level data structure : range tree
  - First level tree : Balanced BST T on x co-ord.
  - Second level tree : Balanced BST $T_{assoc}(v)$ for each v to store P(v) [associated structure of v]



Range Tree : example



Building Range Tree

- Build2dRangeTree(P)
- construct T_{assoc} of points in P on y co-ord. Leaves stores the point and not the y co-ords.
- if P has one point create leaf v along with T_{assoc} (v)
- else split P to P_{left} and P_{right} based on median x co-ord.
  - v_{left} =Build2dRangeTree(P_{left})
  - v_{right} =Build2dRangeTree(P_{right})
  - create v storing median x , make v_{left} and v_{right} left and right child of v, attach T_{assoc} (v)
- return v

Example: Range Tree

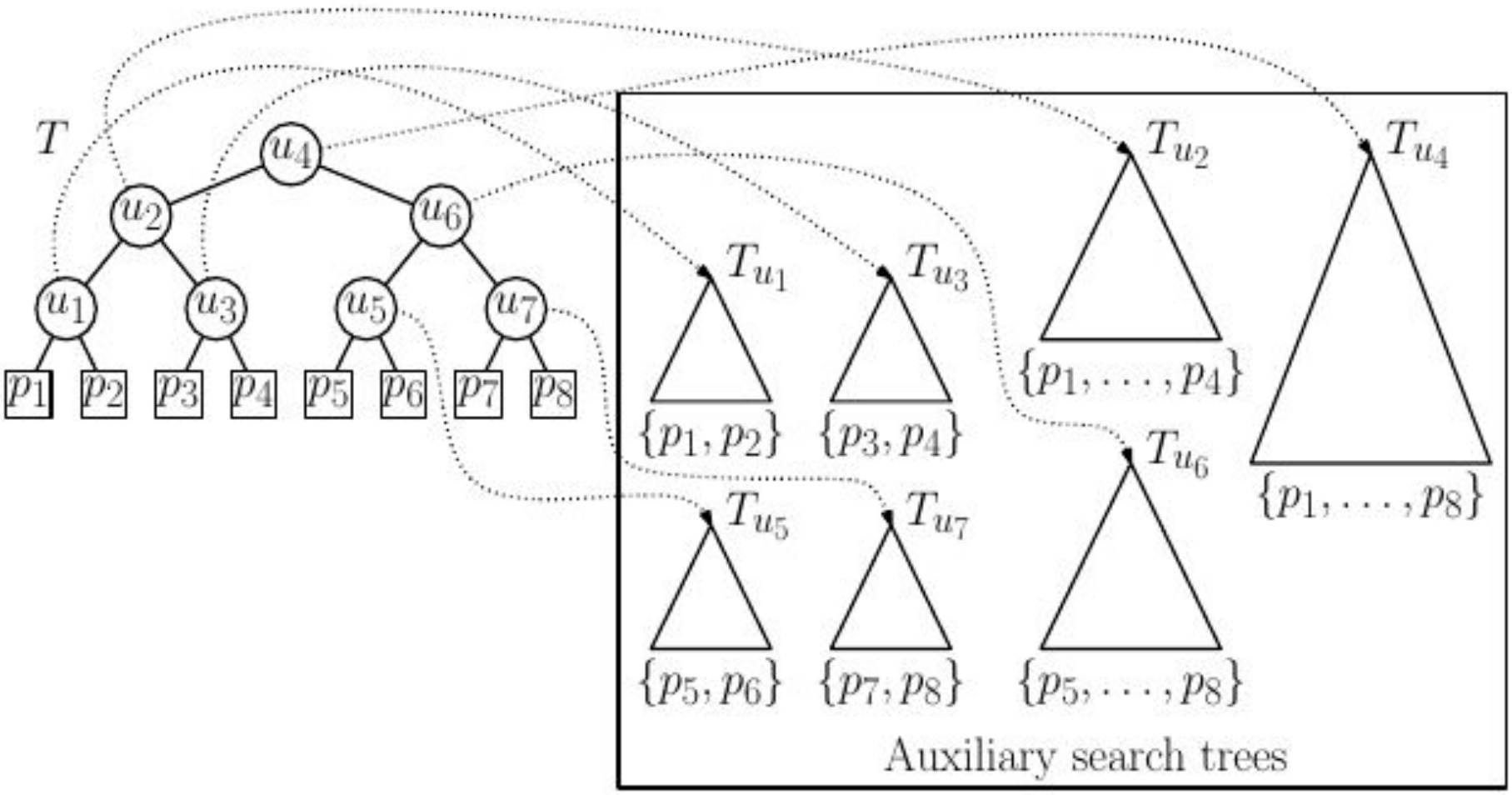
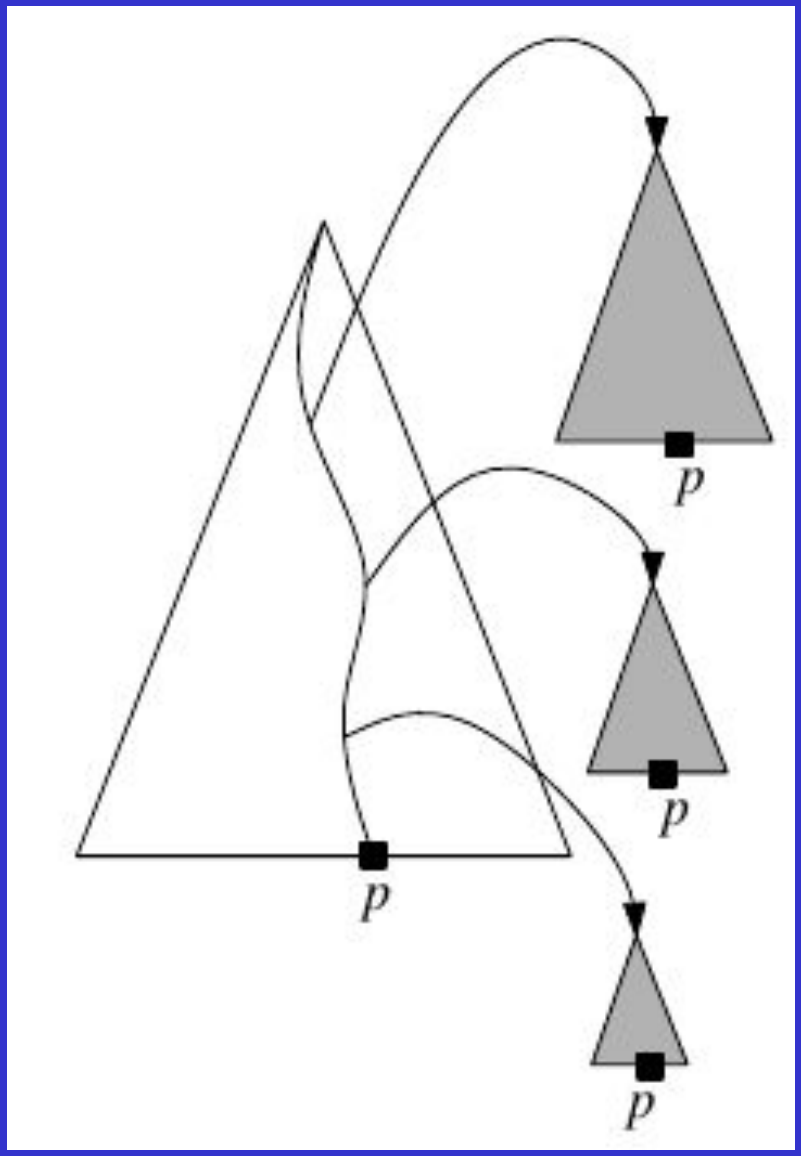


Fig. 158: Multi-level search trees.

Building Range Tree : Space



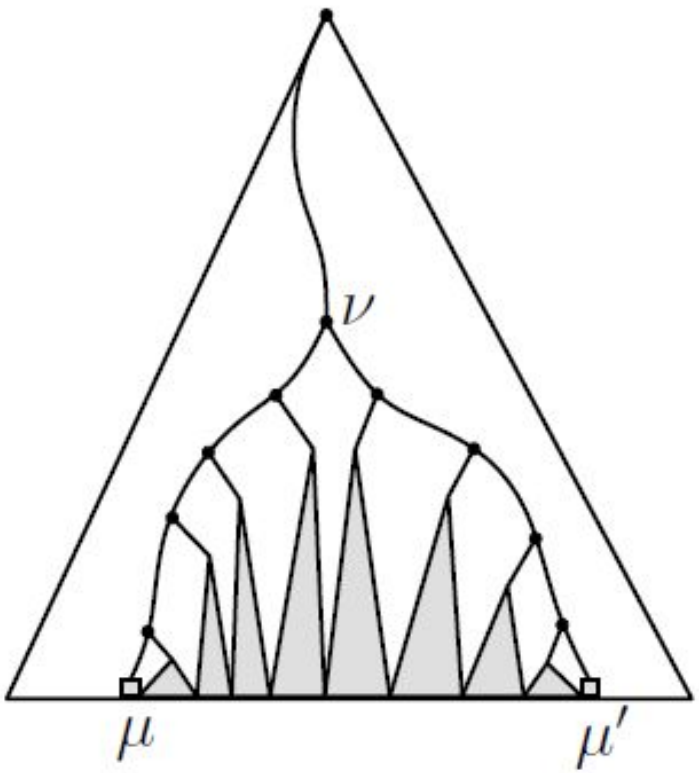
- At any depth of T a pt. is stored only once in $T_{assoc}(v)$,
- At any depth of T , storage : O(n)
- The depth of T : O(log n)
- Total space: O(n log n)

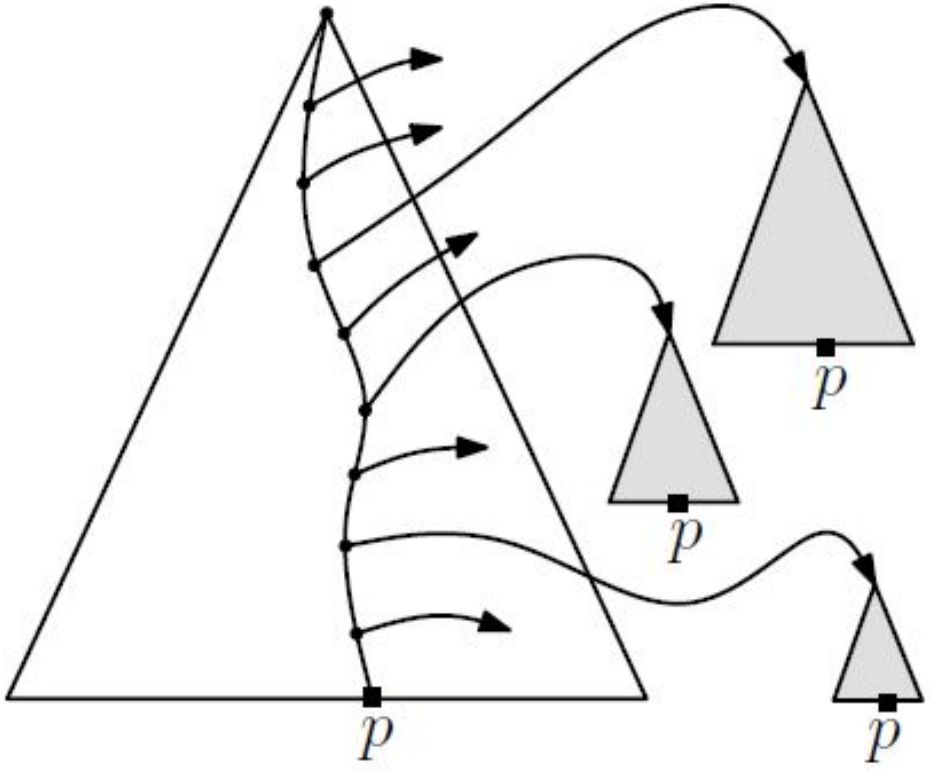
Building Range Tree : Time

- $T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$
  - construction of $T_{assoc}(v)$ takes $O(n \log n)$
- Better construction possible?
  - Presort P on x co-ord. $O(n \log n)$
  - Presort P on y co-ord. $O(n \log n)$
  - Construction of BST $T_{assoc}(v)$: $O(n)$ [since sorted on y, use bottom-up Merging]

- Time Complexity : $O(n \log n)$

Query Algorithm : 2DRangeQuery





O(log n) canonical subsets

Query Algorithm : 2DRangeQuery

- Input: 2 D range tree, $[x_l : x_h][y_l : y_h]$
- Output: All points in $[x_l : x_h][y_l : y_h]$

Vsplit = FindSplitNode(T, x_l , x_h)

if Vsplit is leaf check if this point has to be reported

else

v = lc(Vsplit)

while v is not leaf

if $x_l <= x_v$ then 1DRangequery(T_{assoc}(rc(v)), $[y_l : y_h]$)

v = lc(v)

else v = rc(v)

Check if v is to be reported

Do the same for path from rc(Vsplit) to x_h

Query Algorithm : Example

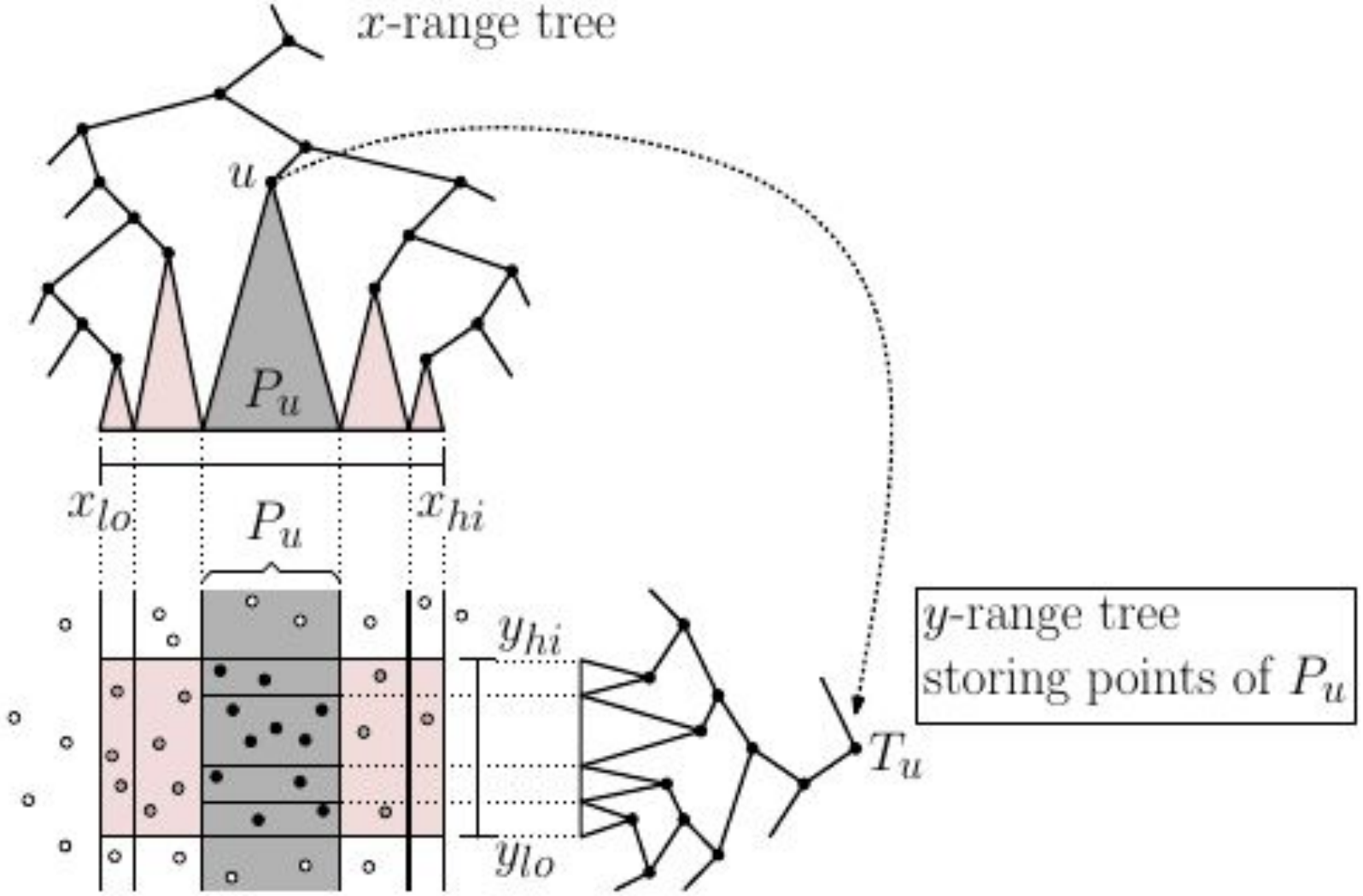
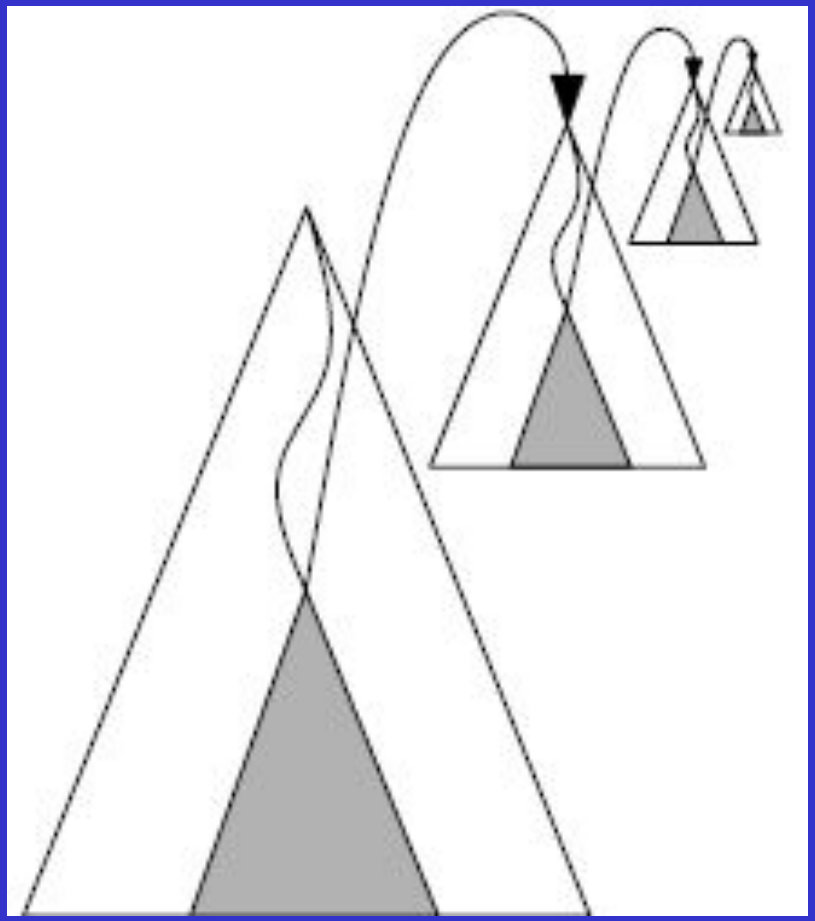


Fig. 159: Orthogonal range tree search.

Query Algorithm : Time Complexity

- At each v, decide which path to follow: O(1) ,
  - call 1DRangequery() on Tassoc (v) : $O(\log n + k_v)$, k_v: # of points reported for v
- Total : $\sum_v O(\log n + k_v) = \sum_v O(\log n) + \sum_v k_v$
  $= O(\log^2 n) + k$
  - $\sum_v O(\log n)$: the length of search path in T is O(log n)
  - $k = \sum_v k_v$: total number of reported points
- Time complexity : $O(\log^2 n + k)$

Higher Dimensional Range Trees



- $p= (x_1,x_2,x_3, \ldots x_d)$ belongs to P
- v has $T_{assoc}(v)$: $(d-1)$ dimensional range tree for P(v)
- Each node in $T_{assoc}(v)$ points to a $(d-2)$ dimensional range tree
- …..
- leaf of last level stores the actual points

Construction time: $O(n \log^{d-1} n)$

Space compexity: $O(n \log^{d-1} n)$

Query time: $O(\log^d n + k)$